

Policies

- Due 11:59 PM PST, January 20th on Gradescope.
- You are free to collaborate on all of the problems, subject to the collaboration policy stated in the syllabus.
- In this course, we will be using Google Colab for code submissions. You will need a Google account.
- You are allowed to use up to 48 late hours across the entire term. Late hours must be used in units of whole hours. Specify the total number of hours you have used when submitting the assignment.
- Students are expected to complete homework assignments based on their understanding of the course material. Student can use LLMs as a resource (e.g., helping with debugging, or grammar checking), but the assignments (including code) should be principally authored by the student.

Submission Instructions

- Submit your report as a single .pdf file to Gradescope (entry code WNB8V4), under "Problem Set 2".
- In the report, **include any images generated by your code** along with your answers to the questions.
- Submit your code by **sharing a link in your report** to your Google Colab notebook for each problem (see naming instructions below). Make sure to set sharing permissions to at least "Anyone with the link can view". **Links that can not be run by TAs will not be counted as turned in.** Check your links in an incognito window before submitting to be sure.
- For instructions specifically pertaining to the Gradescope submission process, see https://www.gradescope.com/get_started#student-submission.

Google Colab Instructions

For each notebook, you need to save a copy to your drive.

1. Open the github preview of the notebook, and click the icon to open the colab preview.
2. On the colab preview, go to File → Save a copy in Drive.
3. Edit your file name to "lastname_firstname_set-problem", e.g. "yue.yisong_set2_prob1.ipynb"

Link with my code solutions

https://drive.google.com/drive/folders/17thyy1M4Z22fLeVUujGAN_G4XPqgS0Fg

I submitted my homework 1/21/2026 at 11:55 PM PST.

1 Comparing Different Loss Functions [30 Points]

Relevant materials: lecture 3 & 4

We've discussed three loss functions for linear classification models so far:

- Squared loss: $L_{\text{squared}} = (1 - y\mathbf{w}^T \mathbf{x})^2$
- Hinge loss: $L_{\text{hinge}} = \max(0, 1 - y\mathbf{w}^T \mathbf{x})$
- Log loss: $L_{\text{log}} = \ln(1 + e^{-y\mathbf{w}^T \mathbf{x}})$

where $\mathbf{w} \in \mathbb{R}^n$ is a vector of the model parameters, $y \in \{-1, 1\}$ is the class label for datapoint $\mathbf{x} \in \mathbb{R}^n$, and we're including a bias term in $\mathbf{x}$ and $\mathbf{w}$. The model classifies points according to $\text{sign}(\mathbf{w}^T \mathbf{x})$.

Performing gradient descent on any of these loss functions will train a model to classify more points correctly, but the choice of loss function has a significant impact on the model that is learned.

Problem A [3 points]: Squared loss is often a terrible choice of loss function to train on for classification problems. Why?

Solution A: *Squared loss is often a terrible choice for classification because:*

1. **It penalizes "too correct" predictions.** Given $m = y\mathbf{w}^T \mathbf{x}$, If a point is classified correctly with a large margin, say $m = 12$, then

$$L_{\text{squared}} = (1 - 12)^2 = 121,$$

which is huge even though the point is perfectly classified.

2. **It spends effort on points that do not matter.** In classification, the points that matter most are near the decision boundary where $m \approx 0$. But squared loss gives large gradients to points far from the boundary. The gradient is

$$\nabla_{\mathbf{w}} L_{\text{squared}} = -2(1 - y\mathbf{w}^T \mathbf{x}) y \mathbf{x},$$

which is proportional to the distance from the decision boundary. The model spends effort adjusting weights to reduce loss on points already correctly classified with large margins, which does not help classification accuracy.

3. **Not robust to outliers.** Outliers can have a large impact on squared loss since they produce very large loss values (quadratic in the margin). A single outlier far from the boundary can dominate the total loss and distort the learned decision boundary.

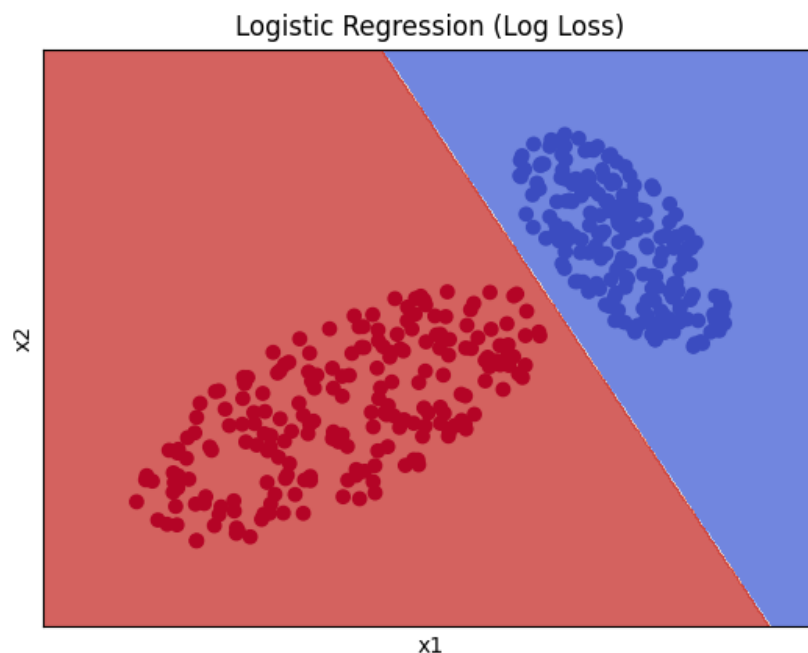
Problem B [9 points]: A dataset is included with your problem set: `problem1data1.txt`. The first two columns represent x_1, x_2 , and the last column represents the label, $y \in \{-1, +1\}$.

On this dataset, train both a logistic regression model and a ridge regression model to classify the points. (In other words, on each dataset, train one linear classifier using $L_{\log}$ as the loss, and another linear classifier using L_{squared} as the loss.) For this problem, you should use the logistic regression and ridge regression implementations provided within scikit-learn ([logistic regression documentation](#)) ([Ridge regression documentation](#)) instead of your own implementations. Use the default parameters for these classifiers except for setting the regularization parameters so that very little regularization is applied.

For each loss function/model, plot the data points as a scatter plot and overlay them with the decision boundary defined by the weights of the trained linear classifier. Include both plots in your submission. The template notebook for this problem contains a helper function for producing plots given a trained classifier.

What differences do you see in the decision boundaries learned using the different loss functions? Provide a qualitative explanation for this behavior.

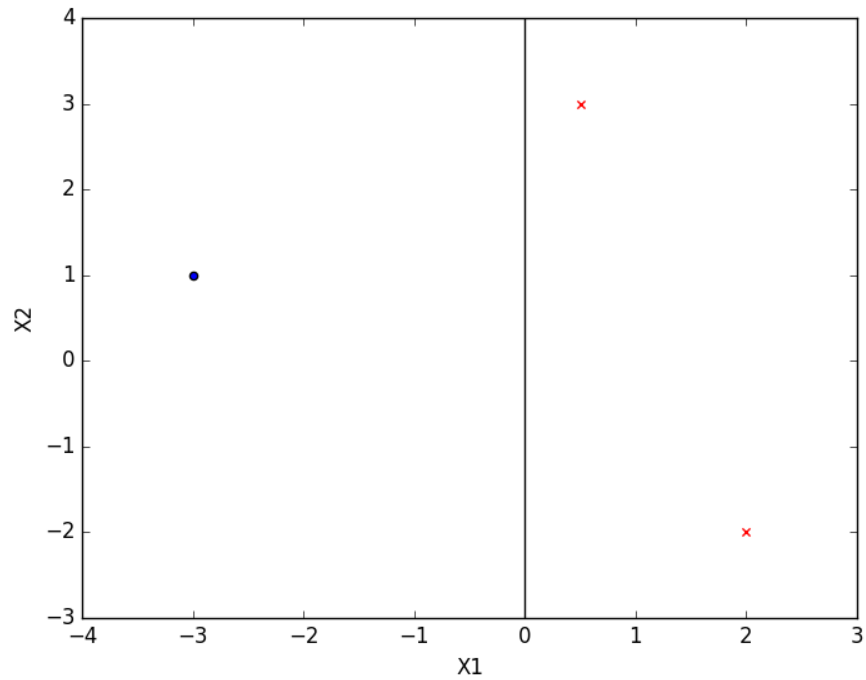
Solution B: *The logistic regression boundary separates all points correctly, while the ridge regression boundary misclassifies some points. This happens because the squared loss penalizes points with large margins. The red points far from the boundary contribute high loss under squared loss, pulling the decision boundary closer to toward them and away from the optimal separating position. Log loss, on the other hand, assigns near-zero loss to confidently correct points, allowing the boundary to focus on actual separation.*





Problem C [9 points]: Leaving squared loss behind, let's focus on log loss and hinge loss. Consider the set of points $S = \{(\frac{1}{2}, 3), (2, -2), (-3, 1)\}$ in 2D space, shown below, with labels $(1, 1, -1)$ respectively.

Given a linear model with weights $w_0 = 0, w_1 = 1, w_2 = 0$ (where w_0 corresponds to the bias term), compute the gradients $\nabla_w L_{\text{hinge}}$ and $\nabla_w L_{\text{log}}$ of the hinge loss and log loss, and calculate their values for each point in S .



The example dataset and decision boundary described above. Positive instances are represented by red x's, while negative instances appear as blue dots.

Solution C: Given:

$$S = \left\{ \left(\frac{1}{2}, 3 \right), (2, -2), (-3, 1) \right\}, \quad y = (1, 1, -1).$$

We include a bias term, so we use the augmented feature vector

$$\tilde{x} = (1, x_1, x_2) \in \mathbb{R}^3, \quad w = (w_0, w_1, w_2).$$

The weights are

$$w_0 = 0, \quad w_1 = 1, \quad w_2 = 0.$$

Hence the score is

$$w^\top \tilde{x} = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 = x_1,$$

and the margin is

$$m = y(w^\top \tilde{x}) = y x_1.$$

Gradients

Hinge loss.

$$L_{\text{hinge}} = \max(0, 1 - m).$$

A subgradient with respect to w is

$$\nabla_w L_{\text{hinge}} = \begin{cases} -y\tilde{x} & \text{if } m < 1, \\ 0 & \text{if } m > 1. \end{cases}$$

Log loss.

$$L_{\text{log}} = \ln(1 + e^{-m}).$$

The gradient with respect to w is

$$\nabla_w L_{\text{log}} = -\frac{y\tilde{x}}{1 + e^m}.$$

Pointwise computations

1) $x = (\frac{1}{2}, 3), y = 1.$

$$\tilde{x} = (1, \frac{1}{2}, 3), \quad w^\top \tilde{x} = \frac{1}{2}, \quad m = yx_1 = 0.5.$$

Since $m < 1$,

$$\nabla_w L_{\text{hinge}} = -y\tilde{x} = -(1, \frac{1}{2}, 3) = (-1, -0.5, -3).$$

Also,

$$\nabla_w L_{\text{log}} = -\frac{(1, \frac{1}{2}, 3)}{1 + e^{0.5}} \approx (-0.37754, -0.18877, -1.13262).$$

2) $x = (2, -2), y = 1.$

$$\tilde{x} = (1, 2, -2), \quad w^\top \tilde{x} = 2, \quad m = yx_1 = 2.$$

Since $m > 1$,

$$\nabla_w L_{\text{hinge}} = 0.$$

Also,

$$\nabla_w L_{\text{log}} = -\frac{(1, 2, -2)}{1 + e^2} \approx (-0.11920, -0.23841, +0.23841).$$

3) $x = (-3, 1), y = -1.$

$$\tilde{x} = (1, -3, 1), \quad w^\top \tilde{x} = -3, \quad m = yx_1 = (-1)(-3) = 3.$$

Since $m > 1$,

$$\nabla_w L_{\text{hinge}} = 0.$$

Also,

$$\nabla_w L_{\log} = -\frac{y\tilde{x}}{1+e^3} = -\frac{(-1)(1, -3, 1)}{1+e^3} = \frac{(1, -3, 1)}{1+e^3} \approx (0.04743, -0.14228, +0.04743).$$

Final gradients

$$\begin{aligned}\nabla_w L_{\text{hinge}}(\tfrac{1}{2}, 3) &= (-1, -0.5, -3), \\ \nabla_w L_{\text{hinge}}(2, -2) &= (0, 0, 0), \\ \nabla_w L_{\text{hinge}}(-3, 1) &= (0, 0, 0),\end{aligned}$$

$$\begin{aligned}\nabla_w L_{\log}(\tfrac{1}{2}, 3) &\approx (-0.37754, -0.18877, -1.13262), \\ \nabla_w L_{\log}(2, -2) &\approx (-0.11920, -0.23841, +0.23841), \\ \nabla_w L_{\log}(-3, 1) &\approx (0.04743, -0.14228, +0.04743).\end{aligned}$$

Problem D [4 points]: Compare the gradients resulting from log loss to those resulting from hinge loss. When (if ever) will these gradients converge to 0? For a linearly separable dataset, is there any way to reduce or altogether eliminate training error without changing the decision boundary?

Solution D: 1. Hinge loss gradients become exactly zero once $m \geq 1$, while log loss gradients decay smoothly and only approach zero as m approaches infinity. 2. Yes, for linearly separable dataset we can reduce the training loss without changing the decision boundary by scaling the weight vector. if we use $c > 1$ to scale the weight vector, this will not change the decision boundary, because $\text{sign}(w^T x) = \text{sign}(cw^T x)$. However, the margin $m = yw^T x$ will be scaled by c as well. Then, for hinge loss, once all margins satisfy $m \geq 1$, the hinge loss will be zero. For log loss, the loss goes to zero asymptotically as m approaches infinity.

Problem E [5 points]: Based on your answer to the previous question, explain why for an SVM to be a “maximum margin” classifier, its learning objective must not be to minimize just L_{hinge} , but to minimize $L_{\text{hinge}} + \lambda \|w\|^2$ for some $\lambda > 0$.

(You don’t need to prove that minimizing $L_{\text{hinge}} + \lambda \|w\|^2$ results in a maximum margin classifier; just show that the additional penalty term addresses the issues of minimizing just L_{hinge} .)

Solution E: Even for separable data, minimizing hinge loss alone is not enough because there are infinitely many classifiers with zero hinge loss (any separator with $y_i w^T x_i \geq 1$ for all i). Among these, some have small margin and some have large margin. Adding the regularizer term $\lambda \|w\|^2$ penalizes large weights, which encourages the model to find a separator with larger margin. Hence minimizing $L_{\text{hinge}} + \lambda \|w\|^2$ leads to a maximum margin classifier.

--

2 Effects of Regularization [40 Points]

Relevant materials: Lecture 3 & 4

For this problem, you are required to implement everything yourself and submit code (i.e. don't use scikit-learn but numpy is fine).

Problem A [4 points]: In order to prevent over-fitting in the least-squares linear regression problem, we add a regularization penalty term. Can adding the penalty term decrease the training (in-sample) error? Will adding a penalty term always decrease the out-of-sample errors? Please justify your answers. Think about the case when there is over-fitting while training the model.

Solution A: Let $SSE(w) = \|y - Xw\|_2^2$. OLS satisfies $w_{LS} \in \arg \min_w SSE(w)$, hence

$$SSE(w_{LS}) \leq SSE(w) \quad \forall w.$$

In particular, taking $w = w_\lambda$ (the ridge solution),

$$SSE(w_{LS}) \leq SSE(w_\lambda),$$

so adding a penalty cannot decrease the in-sample squared error.

Regularization does not always decrease the out of sample error: when the model is overfitting, adding regularization can help reduce overfitting and decrease out of sample error, however, if λ is too large, it can lead to underfitting and increase out of sample error, this is because the test error is a U-shaped function of model complexity.

Problem B [4 points]: ℓ_1 regularization is sometimes favored over ℓ_2 regularization due to its ability to generate a sparse w (more zero weights). In fact, ℓ_0 regularization (using ℓ_0 norm instead of ℓ_1 or ℓ_2 norm) can generate an even sparser w , which seems favorable in high-dimensional problems. However, it is rarely used. Why?

Solution B: When we use $\|w\|_0$ as a regularizer, $\|w\|_0$ counts how many non-zero entries are in w , lots of weights become exactly zero, leading to a sparse solution, it gives the sparsest model. However, optimizing with $\|w\|_0$ is computationally intractable, because the optimization problem becomes combinatorial/ NP-hard, we have to search over subsets of features, and this is not practical for high-dimensional data.

Implementation of ℓ_2 regularization:

We are going to experiment with regression for the Red Wine Quality Rating data set. The data set is uploaded on the course website, and you can read more about it here: <https://archive.ics.uci.edu/>.

[edu/ml/datasets/Wine](https://ml.cs.cmu.edu/ml/datasets/Wine). The data relates 13 different factors (last 13 columns) to wine type (the first column). Each column of data represents a different factor, and they are all continuous features. Note that the original data set has three classes, but one was removed to make this a binary classification problem.

Download the data for training and validation from the assignments data folder. There are two training sets, `wine_training1.txt` (100 data points) and `wine_training2.txt` (a proper subset of `wine_training1.txt` containing only 40 data points), and one test set, `wine_validation.txt` (30 data points). You will use the `wine_validation.txt` dataset to evaluate your models.

We will train a ℓ_2 -regularized logistic regression model on this data. Recall that the unregularized logistic error (a.k.a. log loss) is

$$E = - \sum_{i=1}^N \log(p(y_i|\mathbf{x}_i))$$

where $p(y_i = -1|\mathbf{x}_i)$ is

$$\frac{1}{1 + e^{\mathbf{w}^T \mathbf{x}_i}}$$

and $p(y_i = 1|\mathbf{x}_i)$ is

$$\frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}_i}},$$

where as usual we assume that all $\mathbf{x}_i$ contain a bias term. The ℓ_2 -regularized logistic error is

$$\begin{aligned} E &= - \sum_{i=1}^N \log(p(y_i|\mathbf{x}_i)) + \lambda \mathbf{w}^T \mathbf{w} \\ &= - \sum_{i=1}^N \log \left(\frac{1}{1 + e^{-y_i \mathbf{w}^T \mathbf{x}_i}} \right) + \lambda \mathbf{w}^T \mathbf{w} \\ &= - \sum_{i=1}^N \left(\log \left(\frac{1}{1 + e^{-y_i \mathbf{w}^T \mathbf{x}_i}} \right) - \frac{\lambda}{N} \mathbf{w}^T \mathbf{w} \right). \end{aligned}$$

Implement SGD to train a model that minimizes the ℓ_2 -regularized logistic error, i.e. train an ℓ_2 -regularized logistic regression model. Train the model with 15 different values of λ starting with $\lambda_0 = 0.00001$ and increasing by a factor of 5, i.e.

$$\lambda_0 = 0.00001, \lambda_1 = 0.00005, \lambda_2 = 0.00025, \dots, \lambda_{14} = 61,035.15625.$$

Some important notes: Terminate the SGD process after 20,000 epochs, where each epoch performs one SGD iteration for each point in the training dataset. You should shuffle the order of the points before each epoch such that you go through the points in a random order (hint: use `numpy.random.permutation`). Use a learning rate of 5×10^{-4} , and initialize your weights to small random numbers.

You may run into numerical instability issues (overflow or underflow). One way to deal with these issues is by normalizing the input data X . Given the column for the j th feature, $X_{:,j}$, you can normalize it by setting $X_{ij} = \frac{X_{ij} - \overline{X_{:,j}}}{\sigma(X_{:,j})}$ where $\sigma(X_{:,j})$ is the standard deviation of the j th column's entries, and $\overline{X_{:,j}}$ is the

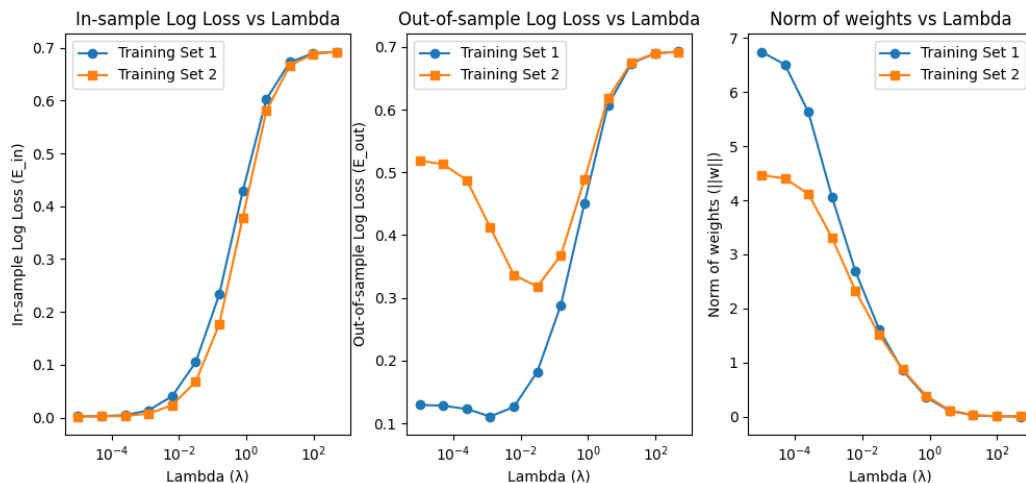
mean of the j th column's entries. Normalization may change the optimal choice of λ ; the λ range given above corresponds to data that has been normalized in this manner. If you treat the input data differently, simply plot enough choices of λ to see any trends.

Problem C [16 points]: Do the following for both training data sets (wine_training1.txt and wine_training2.txt) and attach your plots in the homework submission (use a log-scale on the horizontal axis):

- Plot the average training error (E_{in}) versus different λ s.
- Plot the average test error (E_{out}) versus different λ s using wine_validation.txt as the test set.
- Plot the ℓ_2 norm of $\mathbf{w}$ versus different λ s.

You should end up with three plots, with two series (one for wine_training1.txt and one for wine_training2.txt) on each plot. Note that the E_{in} and E_{out} values you plot should not include the regularization penalty — the penalty is only included when performing gradient descent.

Solution C:



Problem D [4 points]: Given that the data in wine_training2.txt is a subset of the data in wine_training1.txt, compare errors (training and test) resulting from training with wine_training1.txt (100 data points) versus wine_training2.txt (40 data points). Briefly explain the differences.

Solution D: Training on wine_training1 (100 points) produces lower test error and is less sensitive to λ because more data reduces variance and overfitting. Training on wine_training2 (40 points) overfits strongly when λ is small, resulting in much higher test error, and therefore requires a larger λ to generalize well. The training error for both datasets increases as λ increases, but the main difference is the gap between training and test error, which is much larger for the smaller dataset.

Problem E [4 points]: Briefly explain the qualitative behavior (i.e. over-fitting and under-fitting) of the training and test errors with different λ s while training with data in wine_training1.txt.

Solution E: For small λ (weak regularization), the model is very flexible and typically achieves very low training error and also low test error. If λ is extremely small, the model may begin to overfit, meaning the training error remains tiny while the test error stops improving (or may slightly increase). For moderate λ , regularization controls the model complexity and the test error is near its minimum while the training error remains low. For large λ (strong regularization), the weights shrink toward zero and the model becomes too simple, leading to underfitting, where both the training and test errors increase.

Problem F [4 points]: Briefly explain the qualitative behavior of the ℓ_2 norm of $\mathbf{w}$ with different λ s while training with the data in wine_training1.txt.

Solution F: As λ increases, the ℓ_2 norm of the learned weights $\|\mathbf{w}\|_2$ decreases monotonically. For small λ (weak regularization), the penalty is small so the model can use relatively large weights, leading to a larger $\|\mathbf{w}\|_2$. For large λ (strong regularization), the penalty term $\lambda\|\mathbf{w}\|_2^2$ dominates the objective, forcing the optimizer to shrink the weights toward zero, so $\|\mathbf{w}\|_2$ becomes very small.

Problem G [4 points]: If the model were trained with wine_training2.txt, which λ would you choose to train your final model? Why?

Solution G: I would choose the value of λ that minimizes the test (or validation) log-loss, which from the plot occurs at a **moderate** regularization level (roughly $\lambda \approx 10^{-1}$). This is because with fewer training samples the model is more prone to overfitting when λ is too small, and a moderate λ reduces variance and improves generalization, while very large λ would lead to underfitting and higher test error.

3 Lasso (ℓ_1) vs. Ridge (ℓ_2) Regularization [30 Points]

Relevant materials: Lecture 3

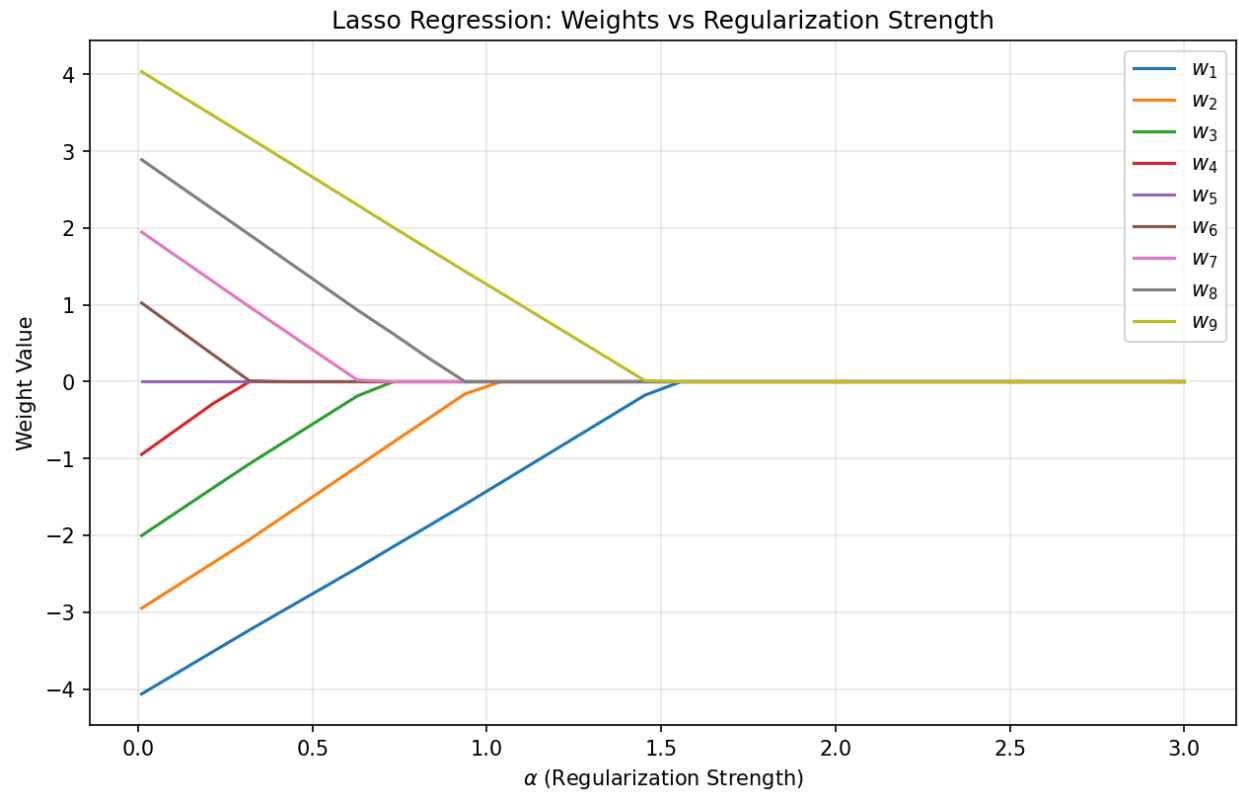
For this problem, you may use the scikit-learn (or other Python package) implementation of Lasso and Ridge regression — you don't have to code it yourself.

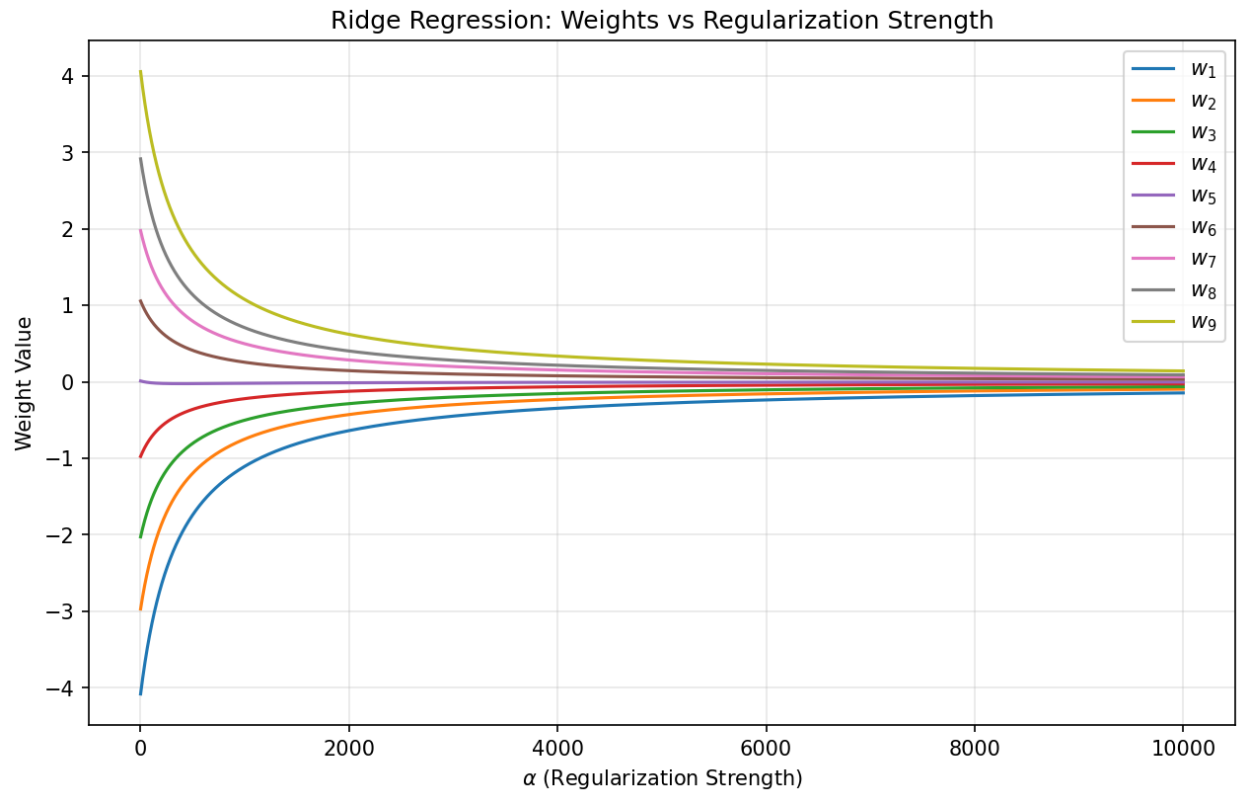
The two most commonly-used regularized regression models are Lasso (ℓ_1) regression and Ridge (ℓ_2) regression. Although both enforce “simplicity” in the models they learn, only Lasso regression results in sparse weight vectors. This problem compares the effect of the two methods on the learned model parameters.

Problem A [12 points]: The tab-delimited file `problem3data.txt` on the course website contains 1000 9-dimensional datapoints. The first 9 columns contain $x_1, \dots, x_9$, and the last column contains the target value y .

- i. Train a linear regression model on the `problem3data.txt` data with Lasso regularization for regularization strengths α in the vector given by `numpy.linspace(0.01, 3, 30)`. On a single plot, plot each of the model weights $w_1, \dots, w_9$ (ignore the bias/intercept) as a function of α .
- ii. Repeat i. with Ridge regression, and this time using regularization strengths $\alpha \in \{1, 2, 3, \dots, 1e4\}$.
- iii. As the regularization parameter increases, what happens to the number of model weights that are exactly zero with Lasso regression? What happens to the number of model weights that are exactly zero with Ridge regression?

Solution A: *Lasso increases the number of exactly-zero coefficients as regularization grows, while Ridge mainly shrinks coefficients continuously without making them exactly zero.*





Problem B [9 points]:

i. In the case of 1-dimensional data, Lasso regression admits a closed-form solution. Given a dataset containing N datapoints, each with $d = 1$ feature, solve for

$$\arg \min_w \|\mathbf{y} - \mathbf{x}w\|^2 + \lambda \|w\|_1,$$

where $\mathbf{x} \in \mathbb{R}^N$ is the vector of datapoints and $\mathbf{y} \in \mathbb{R}^N$ is the vector of all output values corresponding to these datapoints. Just consider the case where $d = 1$, $\lambda \geq 0$, and the weight w is a scalar.

This is linear regression with Lasso regularization.

Solution B.i:

3.B.1

$$\arg \min_w \overbrace{\|y - xw\|^2 + \lambda |w|}^{f(w)}$$

where $x \in \mathbb{R}^N$, $y \in \mathbb{R}^N$

$$\|y - xw\|^2 = (y - xw)^T (y - xw) = y^T y - 2w x^T y + w^2 x^T x$$

$$\text{Then } f(w) = y^T y - 2w(x^T y) + (x^T x)w^2 + \lambda |w|$$

Given that y is not a function of w then

$$\text{we can : } \min_w (x^T x)w^2 - 2w(x^T y) + \lambda |w|$$

$$\text{Let's use } a = x^T x \quad b = x^T y$$

$$\min a w^2 - 2w b + \lambda |w|$$

$$\text{(Case 1) } w > 0 \rightarrow \begin{aligned} g(w) &= a w^2 - 2b w + \lambda w \\ g'(w) &= 2a w - 2b + \lambda = 0 \end{aligned}$$

$$\boxed{w^* = \frac{b - \lambda/2}{a}} \Leftrightarrow b > \lambda/2$$

$$\text{(Case 2) } w < 0 \rightarrow \begin{aligned} g(w) &= a w^2 - 2b w - \lambda w \\ g'(w) &= 2a w - 2b - \lambda = 0 \end{aligned}$$

$$\boxed{w^* = \frac{b + \lambda/2}{a}} \Leftrightarrow b < -\lambda/2$$

$$\text{(Case 3) } w = 0 \rightarrow \begin{aligned} g(w) &= 0 \quad \Leftrightarrow \quad -\lambda/2 \leq b \leq \lambda/2 \end{aligned}$$

ii. In this question, we continue to consider Lasso regularization in 1-dimension. Now, suppose that $w \neq 0$ when $\lambda = 0$. Does there exist a value for λ such that $w = 0$? If so, what is the smallest such value?

Solution B.ii:

3.b.ii $\rightarrow$ Does there exist a value for λ such that $w=0$?

$w \neq 0 \iff |b| \geq \lambda/2$ then

$|b| = \lambda/2 \Rightarrow \boxed{\lambda_{\min} = 2|b| = 2|x^T y|}$

Problem C [9 points]:

i. Given a dataset containing N datapoints each with d features, solve for

$$\arg \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_2^2$$

where $\mathbf{X} \in \mathbb{R}^{N \times d}$ is the matrix of datapoints and $\mathbf{y} \in \mathbb{R}^N$ is the vector of all output values for these datapoints. Do so for arbitrary d and $\lambda \geq 0$.

This is linear regression with Ridge regularization.

Solution C.i:

3. C. i

$$\arg \min ||y - Xw||^2 + \lambda ||w||_2^2 \quad (\lambda \geq 0)$$
$$||y - Xw||^2 = (y - Xw)^T (y - Xw)$$
$$= y^T y - 2y^T Xw + w^T X^T X w$$
$$\lambda ||w||_2^2 = \lambda w^T w$$
$$f(w) = y^T y - 2y^T Xw + w^T X^T X w + \lambda w^T w$$
$$\nabla_w f(w) = -2X^T y + 2X^T X w + 2\lambda w = 0$$
$$X^T X w + \lambda w = X^T y$$
$$(X^T X + \lambda I) w = X^T y$$
$$w^* = (X^T X + \lambda I)^{-1} (X^T y)$$

ii. In this question, we consider Ridge regularization in 1-dimension. Suppose that $w \neq 0$ when $\lambda = 0$. Does there exist a value for $\lambda > 0$ such that $w = 0$? If so, what is the smallest such value?

Solution C.ii:

3.c.i.i.

$$\text{For } w(\lambda) = 0 \rightarrow (X^T X + \lambda I)^{-1} (X^T y) = 0$$

then $X^T y = 0$ ←

but if we suppose $w \neq 0$ when $\lambda = 0$
when $\lambda = 0$

$$w(0) = \frac{X^T y}{X^T X}$$

so $w(0) = 0$ implies $X^T y \neq 0$ → ←

therefore there is no $\lambda > 0$ such that ridge
makes $w = 0$ exactly